

ОБ'ЄКТНО-ОРІЄНТОВАНИЙ ПІДХІД У JAVASCRIPT

Тема 03

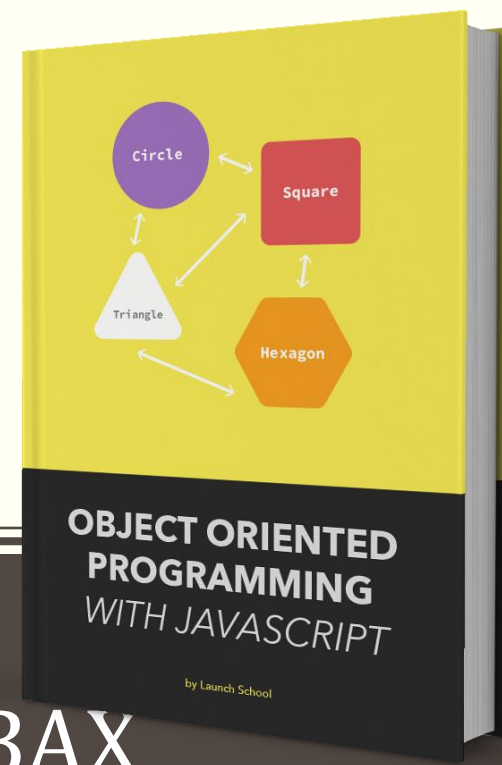
Web-технології та web-дизайн

План лекції

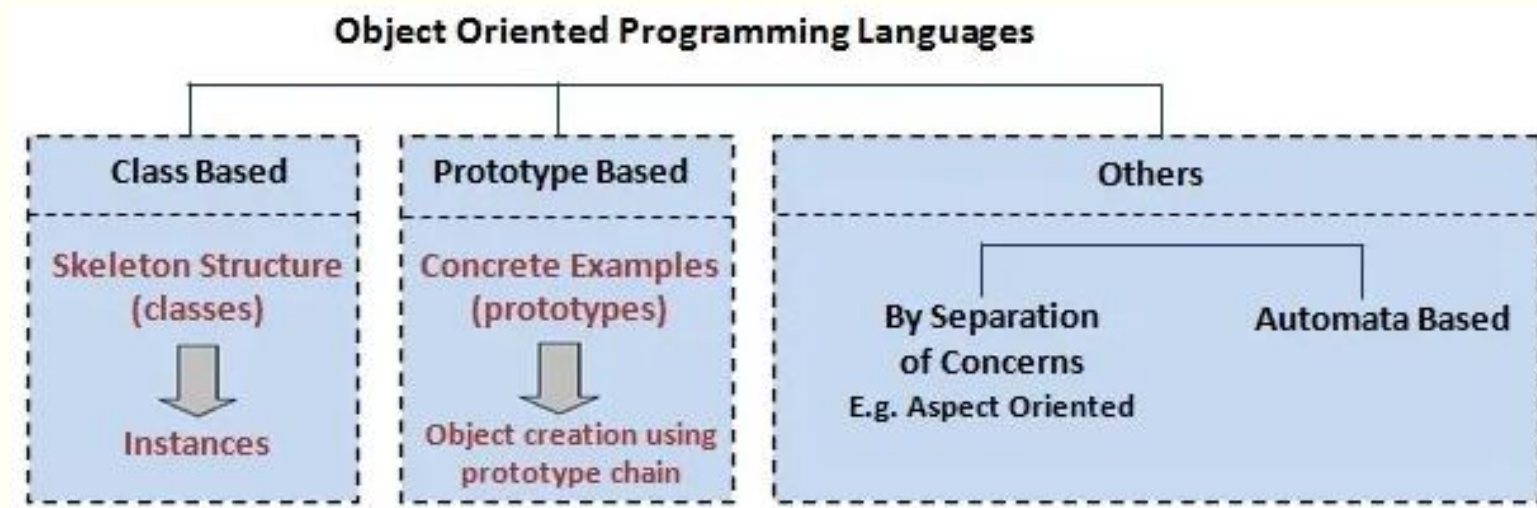
- 1) Принципи імплементації ООП в мовах програмування.
- 2) Технічне порівняння JavaScript і TypeScript.
- 3) Об'єктно-орієнтоване програмування в клієнтських вебзастосунках.
 - 1) <https://dev.to/raboegila/oop-typescript-vs-javascript-4p78>

ПРИНЦИПИ ІМПЛЕМЕНТАЦІЇ ООП В МОВАХ ПРОГРАМУВАННЯ

Питання 3.1.



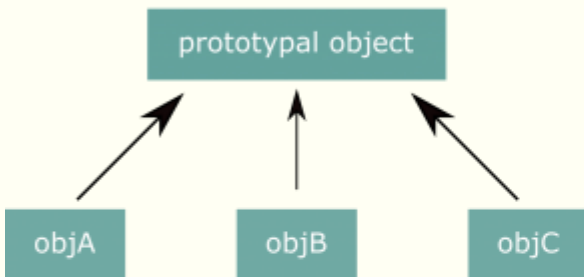
Принципи імплементації ООП в мовах програмування



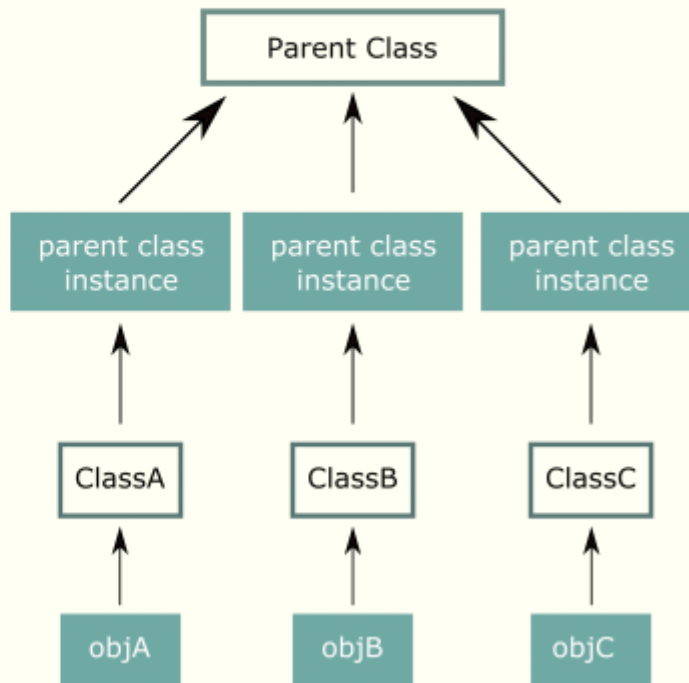
- Мови, засновані на класах, та мови, засновані на прототипах, відрізняються головним чином тим, як вони *створюють групи об'єктів зі спільною поведінкою*.
- Ключові області розбіжностей:
 - 1) створення об'єктів,
 - 2) ієрархічні структури,
 - 3) як об'єкти діляться поведінкою один з одним
 - 4) створення або видалення властивостей об'єкта.
- JavaScript реалізує прототипне ООП.
 - У версії JavaScript ECMAScript 2015 (ES6) було запроваджено class-синтаксис, який дозволяє розробникам використовувати ООП у більш звичний спосіб.

Принципи імплементації ООП в мовах програмування

Prototypal Inheritance



Classical Inheritance



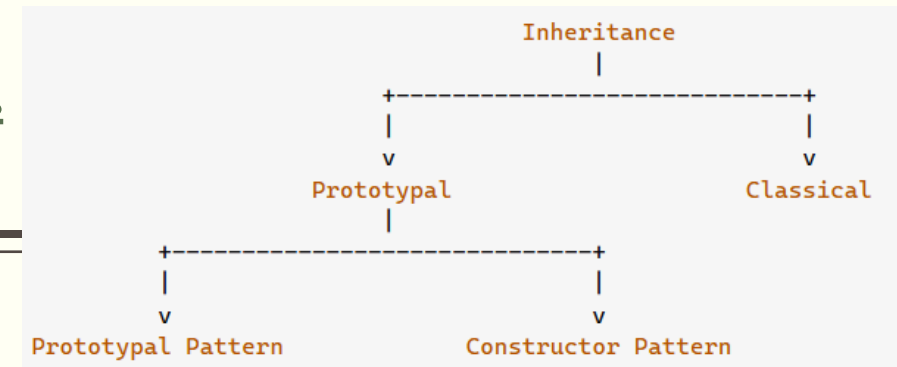
- У класичному успадкуванні ми визначаємо класи, які описують властивості та методи певного типу об'єкта.
 - Потім ми можемо створювати підкласи, які успадковуються від батьківського класу, та додавати або заміщувати (override) деякі властивості чи методи.
 - Це забезпечує чітку ієрархію та структуру для об'єктів, дозволяє повторне використання коду та модульність.
 - Проте таке успадкування може бути жорстким і негнучким, оскільки не дозволяє множинного успадкування або динамічних змін ієрархії класів.
 - Це може створювати тісний зв'язок між підкласами та суперкласами, що ускладнює підтримку та рефакторинг коду, призвести до проблеми крихкості базового класу, коли зміни в суперкласі можуть порушити роботу підкласів.

Порівняння імплементацій у JavaScript.

Класичне ООП

Level of Abstraction	Name of Entity	Comments
0	John Doe	Real World Entity.
1	johnDoe	Variable holding object.
2	Man	Class of object johnDoe.
3	Human	Superclass of class Man.

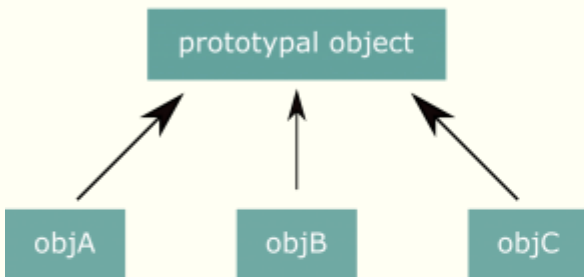
- У класичному ООП ми маємо два типи абстракцій: класи та об'єкти.
 - Об'єкти є лише абстракціями (тобто всі об'єкти мають рівень абстракції 1), а класи є лише узагальненнями (всі класи мають рівень абстракції більше 1).
 - Об'єкти в класичних об'єктно-орієнтованих мовах програмування можна створювати лише шляхом створення екземплярів класів:
 - Зі збільшенням рівня абстракції сутності стають більш загальними, а зі зменшенням рівня абстракції – більш конкретними.



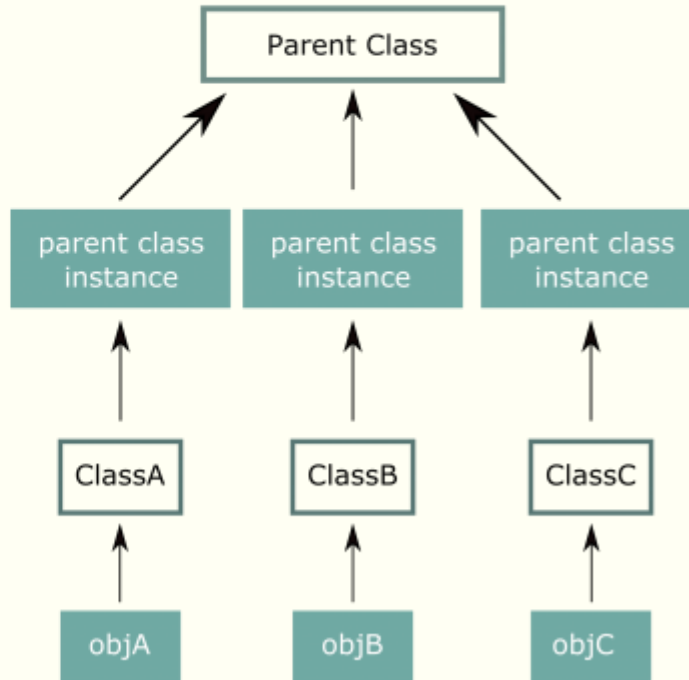
```
class Human {  
    // ...  
}  
  
class Man extends Human {  
    // ...  
}  
  
Man johnDoe = new Man();
```

Принципи імплементації ООП в мовах програмування

Prototypal Inheritance



Classical Inheritance

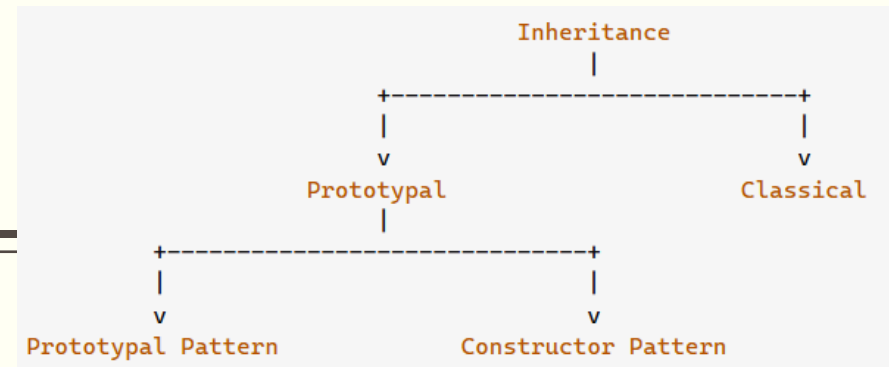


- У прототипному успадкуванні ми не визначаємо класи чи конструктори.
 - Натомість ми створюємо об'єкти безпосередньо з інших об'єктів.
 - Ми можемо використовувати функції або методи для створення нових об'єктів на основі наявних.
 - Таке успадкування є більш гнучким та динамічним, оскільки дозволяє множинне успадкування, домішки (mixin) або композицію об'єктів.
 - Воно більш природне та інтуїтивне зрозуміло, оскільки дотримується принципу «об'єкти як пакети властивостей», має високу виразність, оскільки дозволяє метапрограмування та маніпулювання ланцюжком прототипів.
 - Проте прототипне успадкування може створювати неоднозначність та суперечливість у поведінці та ідентичності об'єкта, здаватись більш заплутаним та незнайомим для розробників.
 - Може з'явитись проблема забруднення прототипу (prototype pollution), коли зміни в прототипі можуть вплинути на всі об'єкти, що успадковуються від нього.

Порівняння імплементацій у JavaScript.

Прототипне ООП

Level of Abstraction	Name of Entity	Comments
0	John Doe	Real World Entity.
1	johnDoe	Variable holding object.
2	man	Prototype of object johnDoe.
3	human	Prototype of object man.



- Прототипні ОО мови програмування набагато простіші за класичні ОО мови програмування, оскільки в прототипному ООП ми маємо лише один тип абстракції – об'єкти.
 - Об'єкти є абстракціями або сутностей реального світу (у цьому випадку їх просто називають об'єктами), або інших об'єктів (у цьому випадку їх називають прототипами тих об'єктів, які вони абстрагують).
 - Отже, прототип є узагальненням.
 - Об'єкти в прототипних ОО мовах програмування можуть бути створені або з нічого, або з іншого об'єкта (який стає прототипом новоствореного об'єкта):

```
var human = {};  
var man = Object.create(human);  
var johnDoe = Object.create(man);
```
 - Класичне успадкування обмежується класами, що успадковуються від інших класів.
 - Однак прототипне успадкування включає не лише прототипи, що успадковуються від інших прототипів, а й об'єкти, що успадковуються від прототипів.

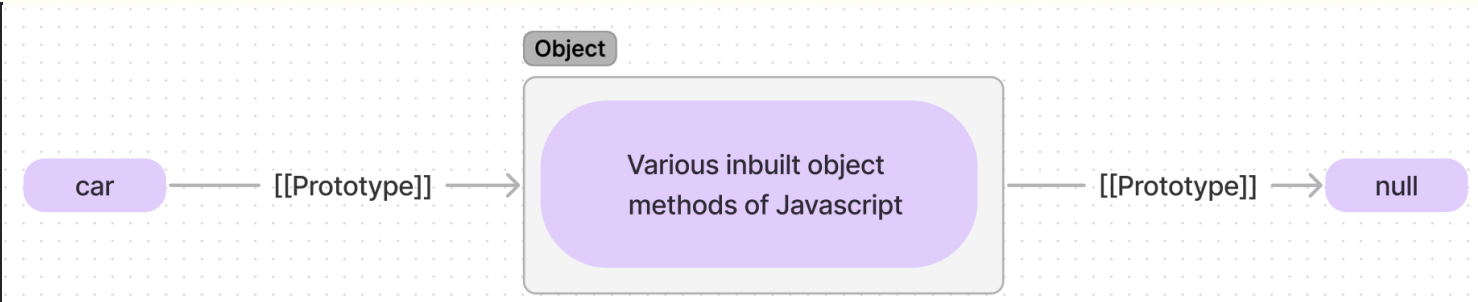
Прототипи в JavaScript

- Коли ми записуємо об'єкт у журнал, ми бачимо всі властивості, визначені нами, а також властивість під назвою `[[Prototype]]`.
 - Кожен об'єкт JavaScript має спеціальну властивість під назвою `[[Prototype]]`, яка вказує на інший об'єкт.
 - У прикладі – на об'єкт-прототип функції-конструктора (constructor function) 'Object'.
 - Саме завдяки прототипу ви можете отримати доступ до властивостей, визначених в інших об'єктах.
 - Це утворює ланцюжок, який триває, доки ми не досягнемо прототипу, прототип якого дорівнює null.

```
> let car = {  
  brand: "Toyota",  
  model: "Camry",  
  year: 2022,  
  drive: function() {  
    console.log("The car is driving.");  
  }  
};
```

```
console.log(car);
```

```
▼ {brand: 'Toyota', model: 'Camry', year: 2022, drive: f} ⓘ  
  brand: "Toyota"  
  ► drive: f ()  
  model: "Camry"  
  year: 2022  
  ► [[Prototype]]: Object
```



Прототипи в JavaScript

- Отримати доступ до прототипу об'єкта за допомогою таких методів:

- `Object.getPrototypeOf()` – повертає прототип зазначеного об'єкта.

```
let prototypeOfCar = Object.getPrototypeOf(car);  
console.log(prototypeOfCar);
```

- Властивість `proto` дозволяє прямий доступ до прототипу об'єкта, проте вважається застарілою. Уникайте в продакшн-коді!

```
let prototypeOfCar = car.__proto__;  
console.log(prototypeOfCar);
```

- Створити прототип для об'єкта можна кількома способами:

- `Object.setPrototypeOf()` - метод для встановлення прототипу заданого об'єкта на інший об'єкт або значення `null`.

```
// Define a prototype object  
let vehicle = {  
  type: 'vehicle',  
  describe: function() {  
    console.log(`This ${this.name} is among the most expensive ${this.type} in India.`);  
  }  
};  
  
// Create a new object 'bike' without a prototype  
let bike = Object.create(null, {  
  name: {  
    value: "Ducati Superleggera V4"  
  }  
});  
  
// Set the prototype of 'bike' to 'vehicle' using Object.setPrototypeOf()  
Object.setPrototypeOf(bike, vehicle);  
  
// Test the prototype relationship  
bike.describe(); // Output: This Ducati Superleggera V4 is among the most expensive vehicle in India.
```

Прототипи в JavaScript

- `Object.create()` – створює новий об'єкт, успадковуючи властивості зазначеного об'єкта.
 - Новий об'єкт успадковує прототип об'єкта, який використовувався для його створення.
- Використання конструктора.
 - Коли ми використовуємо функцію як конструктор, `[[Prototype]]` об'єкта встановлюється як прототип функції, яка використовується як конструктор для створення нового об'єкта.

```
// Define the vehicle
let vehicle = {
  type: 'vehicle',
  describe: function() {
    console.log(`This is a ${this.type} of ${this.brand} manufactured in ${this.year}.`);
  }
};

// Create a new object 'car' based on the 'vehicle'
let car = Object.create(vehicle);
car.brand = 'Toyota';
car.year = 2022;

console.log(car);
car.describe(); // Output: This is a vehicle of Toyota manufactured in 2022.

// Check if the prototype of 'car' is equal to 'vehicle'
console.log(Object.getPrototypeOf(car) === vehicle); // Output: true
```

```
// Define a function 'Car'
function Car(brand, year) {
  this.brand = brand;
  this.year = year;
  this.describe = function() {
    console.log(`This is a ${this.brand} car manufactured in ${this.year}.`);
  }
}

// Create a new object 'myCar' using the 'Car' function as constructor
let myCar = new Car('Toyota', 2022);

// Verify if prototype of myCar is the same as the prototype of Car
console.log(Object.getPrototypeOf(myCar) === Car.prototype); // Output: true
```

Прототипно-класовий ізоморфізм

```
var Human = CLASS(function () {
  var milliseconds = 1
    , seconds      = 1000 * milliseconds
    , minutes      = 60 * seconds
    , hours        = 60 * minutes
    , days         = 24 * hours
    , years        = 365.2425 * days;

  this.constructor = function (name, sex, dob) {
    this.name = name;
    this.sex = sex;
    this.dob = dob;
  };

  this.age = function () {
    return Math.floor((new Date - this.dob) / years);
  };
});

var Man = CLASS(Human, function (Human) {
  this.constructor = function (name, dob) {
    Human.constructor.call(this, name, "male", dob);
    if (this.age() < 18) throw new Error(name + " is a boy, not a man!");
  };
});

var johnDoe = Man.new("John Doe", new Date(1970, 0, 1));
```

- Прототипи та класи настільки схожі, що ви можете використовувати прототипи для моделювання класів:

```
function CLASS(base, body) {
  if (arguments.length < 2) body = base, base = Object.prototype;
  var prototype = Object.create(base, {new: {value: create}});
  return body.call(prototype, base), prototype;

  function create() {
    var self = Object.create(prototype);
    return prototype.hasOwnProperty("constructor") &&
      prototype.constructor.apply(self, arguments), self;
  }
}
```

- Однак зворотне – неправильно, ви не можете використовувати класи для моделювання прототипів.
- Це тому, що прототипи є об'єктами, а класи – ні.
- Вони є зовсім іншим типом абстракції.

Прототипи в JavaScript

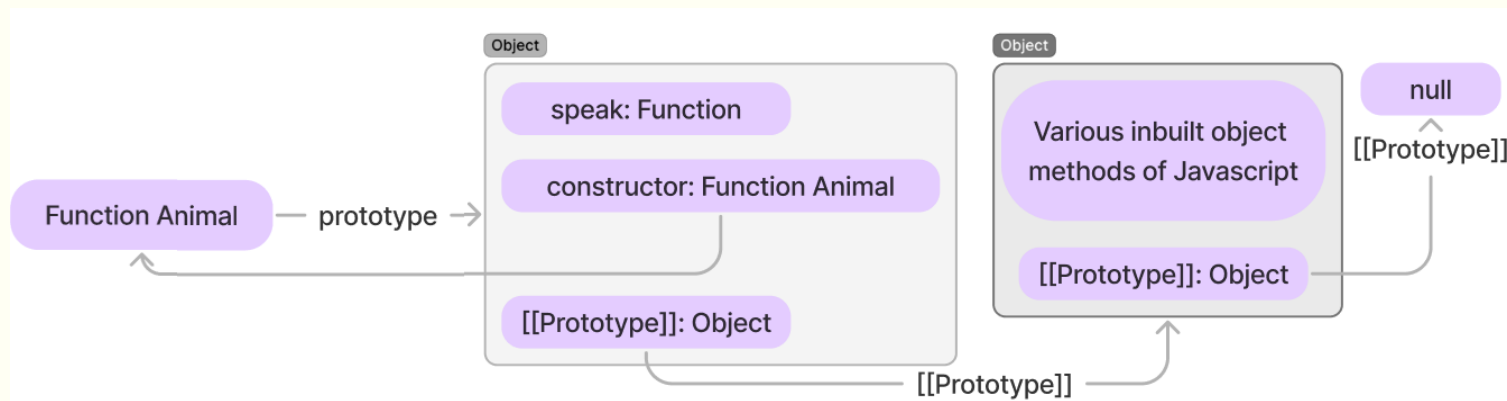
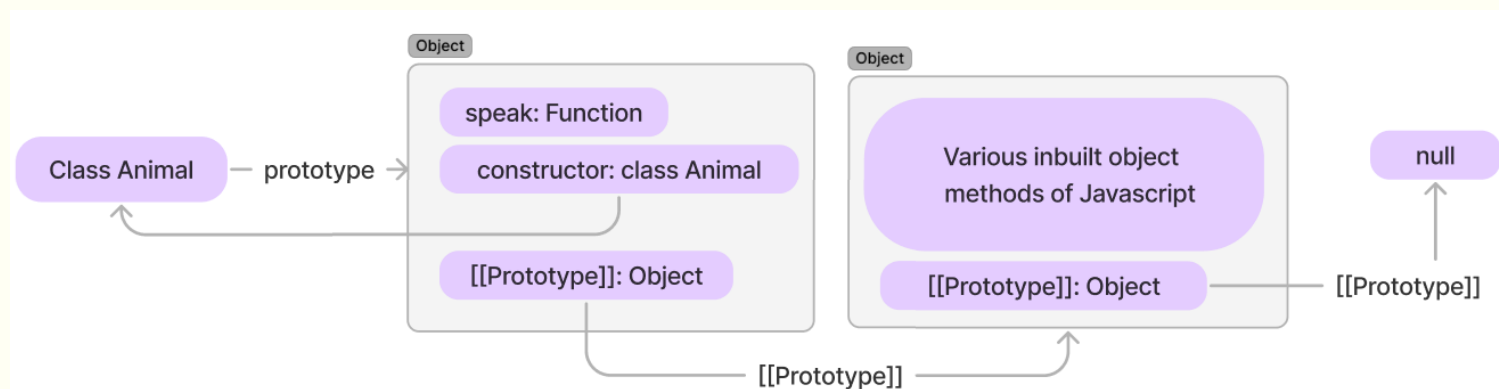
- Класи – це, по суті, синтаксичний цукор над функціями-конструкторами.
 - Коли ви оголошуєте клас у JavaScript, рушій JavaScript автоматично перетворює його на функцію-конструктор та синтаксис на основі прототипів.
 - Класи в JavaScript не є справжніми класами, як у Java або C++, а радше іншим синтаксисом для досягнення подібної функціональності.

```
// Class declaration
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    return this.name + ' makes a noise';
  }
}
```

```
// Equivalent constructor function and prototype
function Animal(name) {
  this.name = name;
}

Animal.prototype.speak = function() {
  return this.name + ' makes a noise';
};
```



Прототипи в JavaScript

```
// Parent class
class Animal {
  constructor(name) {
    this.name = name;
  }

  sayName() {
    console.log('My name is ' + this.name);
  }
}

// Child class inheriting from parent class
class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }

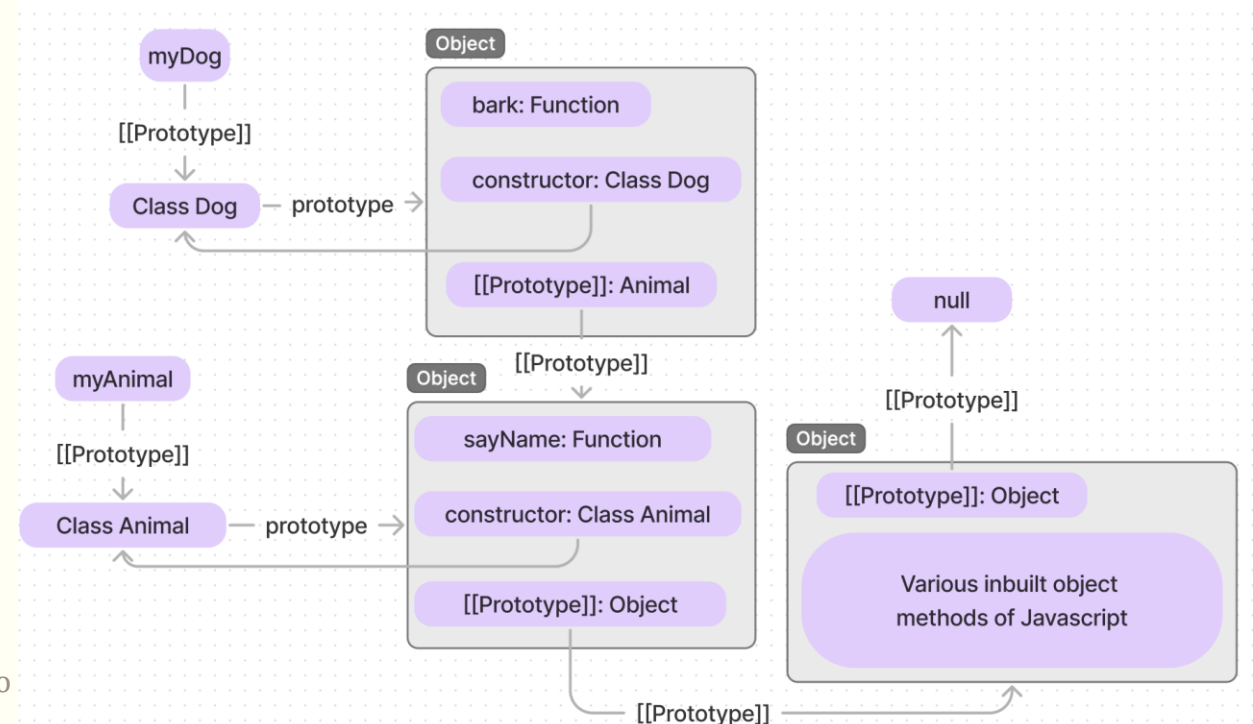
  bark() {
    console.log('Woof! Woof!');
  }
}

// Create instances
const myAnimal = new Animal('Animal');
const myDog = new Dog('Buddy', 'Golden Retriever');

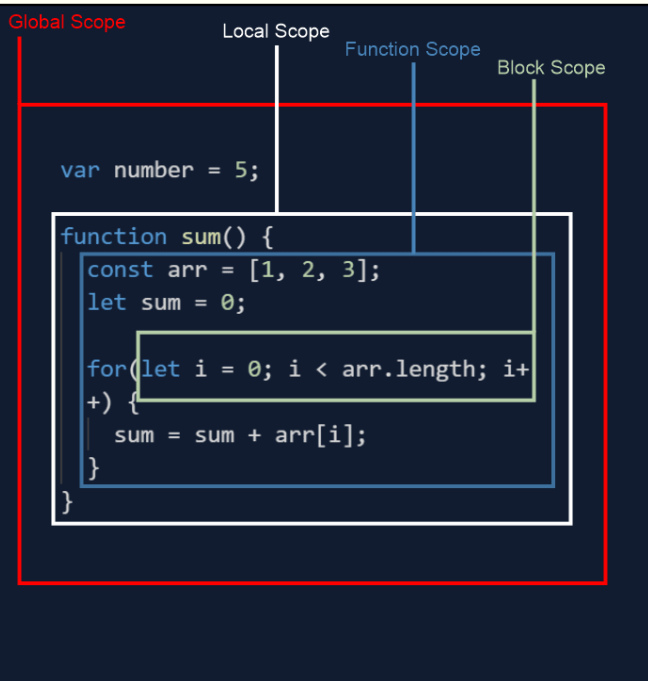
// Call methods
myAnimal.sayName(); // Output: My name is Animal
myDog.sayName(); // Output: My name is Buddy
myDog.bark(); // Output: Woof! Woof!

// Prototype chain
console.log(Object.getPrototypeOf(myDog) === Dog.prototype); // true
console.log(Object.getPrototypeOf(Dog.prototype) === Animal.prototype); // true
console.log(Object.getPrototypeOf(Animal.prototype) === Object.prototype); // true
```

- У прикладі клас Dog успадковується від класу Animal.
 - myDog є екземпляром класу Dog.
 - Прототип класу Dog містить власні функції (включаючи функцію-конструктор), а також успадковується від прототипу класу Animal.
 - Таким чином, успадкування в JavaScript досягається класами.



Прототипи в JavaScript. Затінення (shadowing)



- JavaScript використовує лексичні області видимості (lexical scoping), тобто область видимості змінної визначається її позицією у первинному коді.
 - Коли рушій JavaScript зустрічає змінну, він шукає її в поточній області видимості.
 - Якщо не знаходить її, він переходить до зовнішньої області видимості та продовжує пошук, доки не досягне глобальної області видимості.
 - Якщо змінна з однаковим ім'ям існує в кількох областях видимості, пріоритет має внутрішня, затінюючи зовнішні. У прикладі змінна `message` оголошується на 3 різних рівнях: глобальному, функціональному та блочному.
 - Кожне оголошення замінює попереднє в межах своєї області дії.

```
let message = 'Hello, World!';

function greet() {
  let message = 'Hello, JavaScript!';
  if (true) {
    let message = 'Hello, Block Scope!';
    console.log(message); // Output: 'Hello, Block Scope!'
  }
  console.log(message); // Output: 'Hello, JavaScript!'
}

greet();
console.log(message); // Output: 'Hello, World!'
```

Прототипи в JavaScript. Затінення (shadowing)

- Хоча контроль затінювання є легітимною та часто корисною функцією, він може призвести до плутанини та помилок, якщо його не використовувати обережно.
- Рекомендовані практики для ефективного управління затіненням:
 - Використовуйте *описові* назви змінних, що відображають призначення змінної.
 - *Обмежте область видимості* ваших змінних якомога більше: це не лише мінімізує ризик затінення, але й покращує модульність коду та зручність його обслуговування.
 - Уникайте повторного оголошення змінних у вкладених областях видимості: якщо потрібна змінна із зовнішньої області видимості, використовуйте її безпосередньо, а не затінюйте її новим оголошенням.
 - Використовуйте `const` для змінних, які не слід переприсвоювати, а `let` – для змінних, які можуть змінюватися; уникайте використання `var` – його поведінка в області видимості функцій може призвести до неочікуваних проблем із затіненням.
 - Розберіться з замиканнями (closures): функція зберігає доступ до своєї лексичної області видимості, навіть коли виконується поза цією областю видимості.

```
let data = 'global';

function processData() {
  // Avoid re-declaring 'data' here if you need to use the global 'data'
  console.log(data); // Output: 'global'
}

processData();
```

```
function createCounter ( ) {
  let count = 0 ;
  return function ( ) {
    count++;
    console.log ( count );
  };
}

const counter = createCounter ();
counter (); // Вивід: 1
counter (); // Вивід: 2
```

ES6-класи

- Опис ES6-класу можливий через:
 - оголошення класу (Class Declaration)
 - вираз класу (Class Expression)

```
class Person {  
  constructor(name) {  
    this.name = name;  
  }  
}
```

```
const VariableName = class Person {  
  constructor(name) {  
    this.name = name;  
  }  
}
```

- Додавання властивостей та методів до класу:
 - Конструктор дописується як:

```
constructor() {}
```

```
class Person {  
  value = 15;  
  getValue() {  
    console.log("getValue:", this.value);  
  }  
  printMsg() {  
    console.log("printMsg:", this.value);  
    this.getValue();  
  }  
}
```

Ключове слово new в JavaScript

```
function Ball(x) {
  if (!this) {
    console.error("Tried to call 'Ball' without the new keyword.");
    return;
  }

  this.x = x;
  this.y = 5;
  this.speed = 0;
  this.radius = 10;
  this.color = "#ff2626";

  this.update = function(delta) {
    this.speed += 100 * delta;
    this.y += this.speed * delta;

    if (this.y > 150 && this.speed > 0) {
      this.speed = -150;
    }
  };
}

function main() {
  const balls = [new Ball(25), new Ball(45), new Ball(65)];

  // loop
}
```

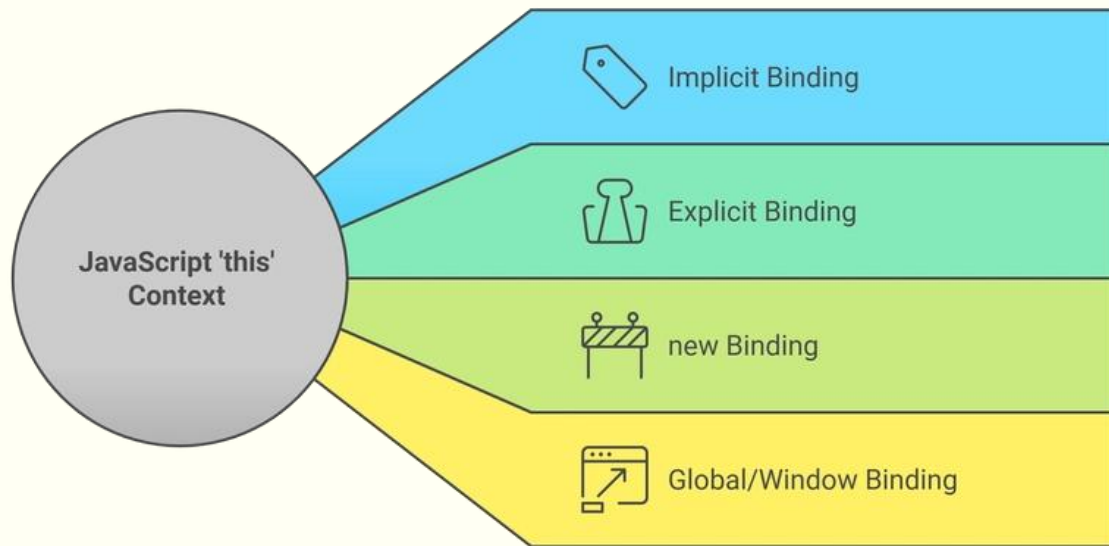
■ Ключове слово new змушує виклик функції діяти подібно до конструктора:

1. Створює порожній об'єкт JavaScript (наприклад, {}).
2. Замінює прототип цього об'єкта на прототип новоствореної функції (функції є об'єктами, і всі об'єкти вже мають прототип).
3. Передає щойно створений об'єкт з кроку 1 як this-змінну (за лаштунками).
4. Повертає значення this, якщо функція не повертає власний об'єкт.

■ Стрілкову функцію не можна використовувати з ключовим словом new.

```
> const Ball = () => { return { speed: 0 } };
< undefined
> new Ball();
✖ ▶ Uncaught TypeError: Ball is not a constructor
   at <anonymous>:1:1
```

Ключове слово this в JavaScript



```
const car = {  
  model: "Tesla",  
  displayModel: function () {  
    console.log(this.model);  
  }  
};  
  
car.displayModel(); // Output: "Tesla"
```

this посилається на об'єкт, який наразі виконує код.

Правила визначення його значення залежать від типу виклику функції:

- Неявне зв'язування: this – це об'єкт, що передує виклику методу. Приклад показано кодом.
- Явне зв'язування: this може бути явно встановлено за допомогою call, apply або bind.
- Прив'язка new: у функціях-конструкторах this посилається на щойно створений об'єкт.
- global/window-зв'язування: якщо жодне з вищезазначених правил не застосовується, this використовується як глобальний об'єкт (window у браузерях або global в Node.js).

Ключове слово `this` в JavaScript. Явне зв'язування

```
function greet(greeting, punctuation) {  
  console.log(`${greeting}, ${this.name}${punctuation}`);  
}  
  
const person = { name: "Jack" };  
  
greet.call(person, "Hello", "!"); // Output: "Hello, Jack!"
```

```
greet.apply(person, ["Hi", "."]); // Output: "Hi, Jack."
```

```
const boundGreet = greet.bind(person, "Welcome");  
boundGreet("?"); // Output: "Welcome, Jack?"
```

Коли вам потрібен точний контроль над значенням `this`, ви можете використовувати методи `call()`, `apply()` або `bind`.

- Метод `call()` негайно викликає функцію із заданим `this`-значенням та окремими аргументами. Тут `call` забезпечує, щоб `this.name` стосувалося властивості `name` об'єкта `person`.
- Метод `apply()` схожий на `call()`, але приймає аргументи як масив, а не окремо.
- Метод `bind()` повертає нову функцію з постійним (permanent) `this`-значенням, встановленим на вказаний об'єкт, що дозволяє відкладений виклик. За допомогою `bind()` ви створюєте функцію повторного використання, прив'язану до певного контексту.

Метод	Призначення	Execution	Передача аргументів
<code>call</code>	Негайний виклик	Так	Окремо
<code>apply</code>	Негайний виклик	Так	Масив аргументів
<code>bind</code>	Відкладений виклик (повертає нову функцію)	Ні	Окремо або частково

Ключове слово this в JavaScript. Прив'язування new та global/window

- Коли функція викликається з ключовим словом new, this посилається на новий об'єкт, що створюється.

```
function Person(name) {  
  this.name = name;  
}  
  
const jack = new Person("Jack");  
console.log(jack.name); // Output: "Jack"
```

- this за замовчуванням використовується глобальний об'єкт.
 - Однак, у строговому режимі ('use strict') у таких випадках this присвоюється значення 'undefined'.
 - Будьте обережні з глобальним зв'язуванням, оскільки це може призвести до небажаної поведінки.

```
function showName() {  
  console.log(this.name);  
}  
  
const name = "Global";  
showName(); // Output: "Global" in non-strict mode
```

Поліморфізм у JavaScript

- Поліморфізм у JS дозволяє використовувати однакову назву методу або властивості для різних об'єктів, і кожен об'єкт реагуватиме або поводитиметься по-різному.
 - Успадкування на основі прототипів дозволяє об'єктам успадковувати властивості та методи від інших об'єктів, а також заміщувати ці властивості та методи.
- Реалізація поліморфізму в JS може здійснюватись кількома способами:
 - перевантаження функцій;
 - заміщення методів;
 - качина типізація.
- **Перевантаження функцій** – це техніка, за якої одна функція може мати кілька сигнатур або способів її виклику.
 - Це дозволяє функції поводитися по-різному залежно від кількості та типів переданих їй аргументів.
 - У контексті мови JavaScript (динамічно та слабо типізованої) фіналізованих сигнатур функції немає – останнє оголошення (definition) функції перезаписує попередні.
 - Тобто, фактично, перевантаження функцій *симулюється!*

```
function greet(name) {  
  console.log(`Hello, ${name}`);  
}
```

```
function greet(firstName, lastName) {  
  console.log(`Hello, ${firstName} ${lastName}`);  
}  
greet("Rafi");  
// Output: "Hello, undefined"
```

Поліморфізм у JavaScript. Способи перевантаження функцій

- Симуляція перевантаження функцій може здійснюватись такими способами:
 - Об'єкт `arguments` (спосіб ES5) дозволяє динамічно отримувати доступ до всіх переданих значень
 - Параметр «решта» (`...args`) в ES6 збирає всі аргументи в масив
 - Перевірки на основі типів
 - Динамічні методи на основі класів

```
function add() {  
  if (arguments.length === 1) {  
    console.log("Square:", arguments[0] * arguments[0]);  
  } else if (arguments.length === 2) {  
    console.log("Sum:", arguments[0] + arguments[1]);  
  } else {  
    console.log("Invalid number of arguments");  
  }  
}
```

```
add(5);      // Square: 25  
add(5, 10);  // Sum: 15
```

```
function calculate(...args) {  
  if (args.length === 1) return args[0] * args[0];  
  if (args.length === 2) return args[0] + args[1];  
  return args.reduce((a, b) => a + b, 0);  
}
```

```
console.log(calculate(5));      // 25  
console.log(calculate(5, 10));  // 15  
console.log(calculate(1, 2, 3, 4)); // 10
```

Поліморфізм у JavaScript. Способи перевантаження функцій

- Перевантаження на основі типів надає вашій функції поліморфної поведінки – різний вивід для різних типів вхідних даних.

- Додатково працюють типові (default) параметри.

```
function greet(name = "Guest") {  
  console.log(`Hello, ${name}!`);  
}
```

- Перевантаження на основі класів полягає в тому, що методи класу можуть легко діяти як перевантажені функції на основі аргументів.

```
function show(value) {  
  if (typeof value === "string") {  
    console.log("String:", value.toUpperCase());  
  } else if (typeof value === "number") {  
    console.log("Number:", value * 2);  
  } else if (typeof value === "object") {  
    console.log("Object:", JSON.stringify(value));  
  } else {  
    console.log("Unsupported type");  
  }  
}
```

```
show("hello");      // String: HELLO  
show(21);           // Number: 42  
show({ name: "Rafi" }); // Object: {"name":"Rafi"}
```

```
class Greeter {  
  greet(...args) {  
    if (args.length === 0) {  
      console.log("Hello, Stranger!");  
    } else if (args.length === 1) {  
      console.log(`Hello, ${args[0]}!`);  
    } else {  
      console.log(`Hello, ${args.join(" ")}!`);  
    }  
  }  
}
```

```
const g = new Greeter();  
g.greet();           // Hello, Stranger!  
g.greet("Rafi");     // Hello, Rafi!  
g.greet("Md", "Rafi"); // Hello, Md Rafi!
```

Поліморфізм у JavaScript. Заміщення методу

```
let person = {
  fName: 'John',
  lName: 'Phillip',
  age: 23,
  getName: function(){
    return `${this.fName} ${this.lName}`
  },
  getAge: function(){
    return `${this.age} years`
  }
}

console.log(person.getName());
console.log(person.getAge());

// create an instance from the person object
let newPerson = Object.create(person);

// instead of a new property, let's add a function with the same name
newPerson.getName = function(){
  return `${this.lName} ${this.fName}`
}

console.log('');
console.log(newPerson.getName());
console.log(newPerson.getAge());
```

- Заміщення методу – це техніка, за якої дочірній об'єкт може перевизначити (override) метод, визначений у його батьківському об'єкті.
 - Це дозволяє дочірньому об'єкту змінювати поведінку методу, щоб краще відповідати власним потребам.
 - Класичний синтаксис на базі прототипів передбачає в прототипі успадкованого об'єкта оголошення функції з такою ж назвою та параметрами, проте іншим тілом.
 - У прикладі прототип newPerson має метод getName(), який є оригінальним методом.
 - Проте тепер newPerson також має свій власний метод під назвою getName().

Поліморфізм у JavaScript. Заміщення методу

```
class Polygon{
  describe(){
    console.log('I am a polygon');
  }
}

class Square extends Polygon{
  describe(){
    console.log('I am a polygon with four equal sides');
  }
}

class Triangle extends Polygon{
  describe(){
    console.log('I am a polygon with three sides');
  }
}

poly = new Polygon();
poly.describe();
// I am a polygon
sq = new Square();
sq.describe();
//I am a polygon with four equal sides

tri = new Triangle();
tri.describe();
// I am a polygon with three sides
```

- Синтаксис на основі класів демонструє Java-стиль заміщень, проте без використання `@Override`.
 - Перевантаження (overloading) методів, на відміну від заміщення, може некоректно працювати в JavaScript.
 - Ми спробували визначити 2 методи з однаковою назвою (`calculateArea()`), але різною кількістю аргументів, і це не спрацювало належним чином:
 - У JavaScript викликається *найновіше* визначення методу.

```
class Polygon{
  describe(){
    console.log('I am a polygon');
  }

  calculateArea(sideA, sideB){
    console.log(sideA*sideB);
  }

  calculateArea(side){
    console.log(side*side);
  }
}

rectangle = new Polygon();
rectangle.calculateArea(5,7);
// 25 (incorrect answer)

square = new Polygon();
square.calculateArea(6);
// 36
```

Поліморфізм у JavaScript. Заміщення методу

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a sound.`);
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }

  // Overriding speak and calling the parent speak method
  speak() {
    super.speak(); // Call the parent class method
    console.log(`${this.name} barks.`);
  }
}

const dog = new Dog('Buddy', 'Golden Retriever');
dog.speak();
// Output:
// Buddy makes a sound.
// Buddy barks.
```

- Іноді може знадобитися перевизначити метод, але все одно викликати реалізацію цього методу батьківського класу.
 - Це можна зробити за допомогою ключового слова `super` у класах ES6, яке дозволяє викликати методи з батьківського класу.
 - У класі `Dog` ми спочатку викликаємо метод `speak` батьківського класу за допомогою `super.speak()`.
 - Це дозволяє нам повторно використовувати загальну поведінку з класу `Animal`, додаючи при цьому специфічну функціональність (гавкіт) у дочірньому класі.

Поліморфізм у JavaScript. Качина типізація

```
function makeQuack(animal) {  
  if (animal.quack) {  
    console.log(animal.quack()); // 🦆🦆  
  } else {  
    console.log("This is not a duck!");  
  }  
}
```

```
const duck = {  
  quack: () => "Quack!",  
};
```

```
const dog = {  
  bark: () => "Woof!",  
};
```

```
const rubberDuck = {  
  quack: () => "Squeak!",  
};
```

```
makeQuack(duck); // Output: "Quack!"  
makeQuack(dog); // Output: "This is not a duck!"  
makeQuack(rubberDuck); // Output: "Squeak!"
```

- Качина типізація – це техніка, за якої об'єкт оцінюється за його поведінкою, а не за типом.
 - Це означає, що об'єкт вважається певного типу, якщо він має необхідні властивості або методи.
 - Качина типізація дозволяє використовувати поліморфізм без явного успадкування.

```
function calculateArea(shape) {  
  if (shape.getArea) {  
    return shape.getArea();  
  }  
  throw new Error("The shape must have a getArea method!");  
}
```

```
const circle = {  
  radius: 5,  
  getArea() {  
    return Math.PI * this.radius ** 2;  
  },  
};
```

```
const rectangle = {  
  width: 10,  
  height: 4,  
  getArea() {  
    return this.width * this.height;  
  },  
};
```

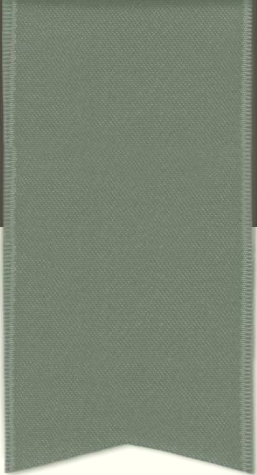
```
console.log(calculateArea(circle)); // Output: 78.53981633974483  
console.log(calculateArea(rectangle)); // Output: 40
```

Поліморфізм у JavaScript. Качина типізація

✅ Переваги	❌ Недоліки
Гнучкість — можна використовувати будь-який об'єкт, який має потрібні методи/властивості, незалежно від його типу.	Помилки під час виконання (runtime) — якщо потрібний метод відсутній, помилка виникне лише під час роботи програми.
Спрощення коду — не потрібно оголошувати строгі типи або ієрархії, менше boilerplate-коду.	Зниження читабельності — без явних типів складніше зрозуміти, які об'єкти очікує функція (особливо у великих проєктах).
Природна підтримка поліморфізму — різні об'єкти можуть оброблятися однаково без наслідування або інтерфейсів.	Складніше дебажити — причина помилки може бути неочевидною через відсутність типових контрактів.
Інтероперабельність — об'єкти з різних бібліотек можуть працювати разом, якщо реалізують потрібну поведінку.	Відсутність compile-time safety — немає гарантій структури об'єкта під час компіляції (на відміну від статично типізованих мов).

- Хоча JavaScript не має явних визначень типів, такі інструменти, як TypeScript, дозволяють розробникам поєднувати переваги качиної типізації зі статичною типізацією.
 - Оголошуючи змінні та параметри функцій в TypeScript, додавання значень за замовчуванням може бути більш практичним захистом від помилок рантайму, ніж тільки визначення типу.
 - Правило “Always Defaults” (обговорення)

```
interface Quackable {  
  quack: () => string;  
}  
  
function makeQuack(animal: Quackable) {  
  console.log(animal.quack());  
}  
  
const duck = {  
  quack: () => "Quack!",  
};  
  
makeQuack(duck); // Output: "Quack!"
```



ДЯКУЮ ЗА УВАГУ!

Наступне питання: Технічне порівняння JavaScript і TypeScript.